

Databases from the Inside

A Complete Beginner-to-Advanced Study Guide: Indexes, Query Planners,
Transactions, and MVCC

July 2026

Contents

1	How to Use This Guide	3
2	Plain-English Glossary (Read This First)	3
3	Part 1: The Physical Layer — How Data Actually Lives on Disk	4
3.1	Pages: the atom of database I/O	4
3.2	The buffer pool: why “hot” data is fast	4
3.3	Why finding things is the hard part	4
4	Part 2: Indexes — The Heart of Performance	4
4.1	The B-tree, explained like a phone book	5
4.2	What B-trees can and cannot accelerate	5
4.3	Composite indexes and the leftmost-prefix rule	5
4.4	Covering indexes and index-only scans	6
4.5	Other index types (know they exist)	6
4.6	The price of indexes	6
5	Part 3: The Query Planner — The Database’s Brain	6
5.1	From SQL to plan	6
5.2	Reading EXPLAIN (the essential skill)	7
5.3	The three join algorithms	7
6	Part 4: Transactions and ACID	7
6.1	What a transaction promises	7
6.2	Durability’s engine: the Write-Ahead Log (WAL)	8
7	Part 5: MVCC — How Readers and Writers Coexist	8
7.1	The problem and the naive answer	8
7.2	The MVCC idea	8
7.3	The bill: dead tuples and VACUUM	9
8	Part 6: Isolation Levels and Their Anomalies	9

9 Part 7: Locks and Deadlocks	10
10 Part 8: Storage Engine Alternatives — B-trees vs. LSM-trees (Advanced)	10
11 Part 9: Common Mistakes, Best Practices, FAQs	10
11.1 Common mistakes	11
11.2 Best practices	11
11.3 FAQs	11
12 Final Cheat Sheet (One-Page Revision)	12

1 How to Use This Guide

Most developers use databases every day but treat them as magic boxes: SQL goes in, rows come out, and sometimes everything is mysteriously slow. This guide opens the box. Once you understand how a database *physically* stores, finds, and protects data, slow queries stop being mysteries, indexes stop being folklore, and words like “deadlock” and “isolation level” become concrete.

We use PostgreSQL as the main reference (with MySQL/InnoDB notes where they differ), because it’s what most developers actually get stuck on. Everything here applies conceptually to almost every relational database.

The one-sentence summary: *A database is a program that arranges data on slow disks so it can be found fast (indexes), decides how to find it (the query planner), and lets many users change it at once without corrupting it (transactions, MVCC, locks, and the write-ahead log).*

2 Plain-English Glossary (Read This First)

- **Page (or block):** The fixed-size unit a database reads/writes from disk — 8 KB in PostgreSQL, 16 KB in InnoDB. Databases never read “a row”; they always read whole pages.
- **Heap:** PostgreSQL’s name for the main table storage — pages full of rows in no particular order.
- **Index:** A separate data structure that maps column values to row locations, so the database can find rows without scanning the whole table.
- **B-tree:** The balanced tree structure behind almost every default index.
- **Query planner / optimizer:** The database component that decides *how* to execute your SQL — which indexes to use, in what order to join tables.
- **Execution plan:** The step-by-step recipe the planner produces (viewable with EXPLAIN).
- **Transaction:** A group of operations that succeed or fail as one unit.
- **ACID:** The four transaction guarantees — Atomicity, Consistency, Isolation, Durability.
- **MVCC (Multi-Version Concurrency Control):** The trick of keeping multiple versions of each row so readers and writers don’t block each other.
- **Snapshot:** The consistent “photograph” of the database a transaction sees while it runs.
- **Isolation level:** How strongly transactions are shielded from seeing each other’s in-progress work.
- **Lock:** A reservation on a row/table preventing conflicting concurrent changes.
- **Deadlock:** Two transactions each waiting for a lock the other holds — forever, unless the database kills one.
- **WAL (Write-Ahead Log):** An append-only journal of every change, written *before* the data files — the foundation of crash safety and replication.
- **VACUUM:** PostgreSQL’s garbage collector for dead row versions left behind by MVCC.

- **Selectivity:** What fraction of rows a condition matches. `WHERE id = 5` is highly selective; `WHERE active = true` may match 95% of the table.
- **Cardinality:** The number of distinct values in a column.

3 Part 1: The Physical Layer — How Data Actually Lives on Disk

3.1 Pages: the atom of database I/O

Everything starts with one fact: **disks are read in pages, not rows**. A table is a file (or set of files) divided into fixed-size pages. Each page holds a header, row data, and a small directory of pointers to the rows inside it.

One 8 KB heap page (PostgreSQL):

```
+-----+
| page header                               |
| item pointers: [->row1] [->row2] [->row3]|
|           (free space in the middle)      |
|           row3 data | row2 data | row1    |
+-----+
```

Consequences you'll feel in practice:

- Reading one 100-byte row costs a full 8 KB page read. Reading 50 rows that happen to live on the same page costs... one page read. **Data locality is everything.**
- A row is addressed by (page number, slot number) — in PostgreSQL this is the hidden `ctid` column. Indexes store these addresses.
- **Narrow tables are fast tables:** more rows fit per page, so every scan and cache slot works harder for you.

3.2 The buffer pool: why “hot” data is fast

Databases keep a large cache of pages in RAM (the *buffer pool* / `shared_buffers`). Every read checks RAM first. A query touching cached (“hot”) pages can be 100–1000x faster than one going to disk. This is why the second run of a query is often dramatically faster, and why “it’s fast in staging, slow in prod” is often just a cache-size story.

3.3 Why finding things is the hard part

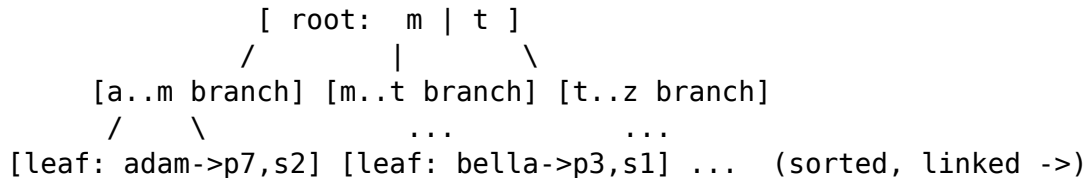
Without help, answering `WHERE email = 'a@b.com'` on a 10-million-row table means reading *every page* — a **sequential scan** (full table scan). At ~100 rows/page that’s 100,000 page reads for one lookup. Fine once; catastrophic at 1,000 queries/second. Enter indexes.

4 Part 2: Indexes — The Heart of Performance

4.1 The B-tree, explained like a phone book

A B-tree index is a balanced tree of pages:

- **Leaf level:** all the indexed values, in sorted order, each with a pointer to its row in the heap. Leaves are linked left-to-right (great for ranges).
- **Internal levels:** “signposts” — each entry says “values up to X are down this branch.”



Because the tree is balanced and each page holds hundreds of keys, even a billion rows need only **3-4 levels**. A lookup reads 3-4 pages instead of millions. That is the entire magic of indexing: it converts $O(N)$ scans into $O(\log N)$ descents.

Analogy: a phone book. Names are sorted (leaf level), and the thumb tabs / headers on each page (internal levels) let you jump straight to the right region. Nobody reads a phone book cover to cover — unless you’re searching by *phone number*, in which case the sorting is useless and you must read everything. That’s exactly a query with no matching index.

4.2 What B-trees can and cannot accelerate

A B-tree on (col) accelerates: =, <, >, BETWEEN, IN, ORDER BY col (data is pre-sorted!), and prefix LIKE 'abc%'.

It **cannot** help with:

- LIKE '%abc' or %abc% (no defined starting point in sorted order),
- conditions on *expressions* of the column: WHERE lower(email) = ... or WHERE created_at + interval '1 day' > ... — the index stores email, not lower(email). Fix: a **functional index** on lower(email), or rewrite the query so the column stands alone.

4.3 Composite indexes and the leftmost-prefix rule

An index on (last_name, first_name, birthdate) is sorted by last name, then first name within it, then birthdate within that — like a phone book sorted by exactly those three things.

The leftmost-prefix rule: the index helps queries that constrain a *prefix* of the columns:

- WHERE last_name = 'Shah' — yes.
- WHERE last_name = 'Shah' AND first_name = 'Priya' — yes, even better.
- WHERE first_name = 'Priya' — **no**. Priyas are scattered throughout the book.

Ordering heuristic: put equality-filtered, high-selectivity columns first; a range-filtered column last (after a range, later columns can’t be used for navigating, only filtering).

4.4 Covering indexes and index-only scans

If the index itself contains every column a query needs (`INCLUDE (...)` in Postgres, or just extra key columns), the database can answer from the index alone — an **index-only scan** — never touching the table. Often the single biggest optimization for hot read queries.

4.5 Other index types (know they exist)

- **Hash**: equality only; rarely worth it over B-tree.
- **GIN** (inverted index): for “contains” questions — full-text search, JSONB keys, array membership.
- **GiST / SP-GiST**: geometric/range data, nearest-neighbor.
- **BRIN**: tiny summaries per block range; superb for huge append-only tables queried by time.
- Vector/ANN indexes (HNSW etc.) power semantic search — see the RAG guide (folder 02).

4.6 The price of indexes

Indexes are not free: every `INSERT/UPDATE/DELETE` must also update every index on the table (write amplification), they consume disk and cache, and unused ones are pure tax. Index the queries you actually run; audit usage (`pg_stat_user_indexes`) and drop dead weight.

5 Part 3: The Query Planner — The Database’s Brain

5.1 From SQL to plan

SQL is *declarative*: you say **what** you want, never **how**. The journey: parse → rewrite (expand views, etc.) → **plan/optimize** → execute. The planner enumerates possible strategies, estimates each one’s cost using **statistics** it keeps about your data (row counts, distinct values, most-common values, histograms — refreshed by `ANALYZE`), and picks the cheapest.

Cost is an abstract blend of estimated page reads and CPU — that’s why the planner cares so much about **selectivity**: fetching 0.1% of a table via an index is cheap; fetching 40% via an index (jumping around the heap page by page) is *slower* than just scanning everything sequentially. **A sequential scan is not automatically bad** — for big fractions of a table it’s the right call.

5.2 Reading EXPLAIN (the essential skill)

EXPLAIN shows the plan; EXPLAIN ANALYZE *runs* the query and shows estimated vs. actual. Read it inner (most-indented) first:

```
EXPLAIN ANALYZE
SELECT * FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE c.country = 'IN' AND o.created_at > now() - interval '7 days';

Nested Loop                                (rows est=1200  actual=980)
  -> Index Scan on customers_country_idx    (est=300  actual=310)
      Index Cond: country = 'IN'
  -> Index Scan on orders_customer_created_idx (est=4  actual=3 per loop)
      Index Cond: customer_id = c.id AND created_at > ...
```

What to look for, in order:

1. **Seq Scan on a big table with a selective WHERE** → missing/unusable index.
2. **Estimated vs. actual rows wildly different** (10x+) → stale or insufficient statistics → run ANALYZE; the planner is flying blind and will pick wrong joins.
3. **The most expensive node** — optimize the biggest cost, not the first line.
4. **Filter vs. Index Cond:** Filter means rows were fetched *then* discarded — work the index could have avoided.
5. **Sort nodes** spilling to disk → maybe an index can provide the order for free.

5.3 The three join algorithms

Every SQL join is executed as one of three physical algorithms; the planner chooses per query:

- **Nested Loop:** for each row of the outer table, look up matches in the inner (ideally via an index). Best when the outer side is *small* and the inner lookup is indexed. Deadly when both sides are big and unindexed (N x M).
- **Hash Join:** build an in-memory hash table of the smaller side, stream the bigger side through it. The workhorse for large equality joins.
- **Merge Join:** if both sides are (or can cheaply be) sorted on the join key, zip them together in one pass. Great with pre-sorted indexes.

You don't pick these — but you *enable* the good ones by providing indexes and honest statistics.

6 Part 4: Transactions and ACID

6.1 What a transaction promises

```
BEGIN;
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

- **Atomicity** — all or nothing. Crash after the first UPDATE? On restart, it's as if nothing happened. No money vanishes mid-transfer.
- **Consistency** — every commit moves the database from one valid state to another; constraints (unique, foreign key, checks) hold.
- **Isolation** — concurrent transactions don't see each other's half-done work (degrees of this below).
- **Durability** — once COMMIT returns, the change survives power loss.

6.2 Durability's engine: the Write-Ahead Log (WAL)

Rule: **log first, data later**. Every change is appended to the WAL and flushed to disk *before* commit returns; the actual table/index pages are updated lazily later (check-points). After a crash, the database replays the WAL to redo committed work and discard uncommitted work.

Why it's brilliant: appending to one sequential log is far faster than scattering random writes across data files — you get crash safety *and* speed. The WAL is also the feed for **replication** (ship the log to replicas) and point-in-time recovery (replay logs up to 09:59, right before the bad deploy).

7 Part 5: MVCC — How Readers and Writers Coexist

7.1 The problem and the naive answer

Thousands of transactions read and write simultaneously. The naive answer — lock everything — means readers block writers and writers block readers, and throughput dies.

7.2 The MVCC idea

Never overwrite in place; keep versions.

- In PostgreSQL, every UPDATE writes a *new row version*; the old one stays, stamped with “created by transaction X, deleted by transaction Y” (hidden columns xmin/xmax). A DELETE just stamps xmax.
- Every transaction gets a **snapshot**: the set of transactions that had committed when it started. It sees exactly the row versions visible to that snapshot — a consistent photograph — no matter what others change meanwhile.

Result: **readers never block writers, and writers never block readers**. Writers only block *other writers touching the same row*.

```
Row "balance" over time:      T=reader (snapshot at 10:00)
v1: 500 (committed 09:58) <-- T sees this
v2: 400 (written 10:01 by another txn, committed 10:02)
T still reads 500 until it ends. New transactions read 400.
```


7.3 The bill: dead tuples and VACUUM

Old versions (“dead tuples”) pile up. PostgreSQL’s **VACUUM** (usually autovacuum) reclaims them once no snapshot can need them, and updates statistics. Practical fallout every backend dev eventually meets:

- **Table bloat:** update-heavy tables grow with dead versions; queries slow because pages are full of garbage. Watch autovacuum health.
- **Long-running transactions are toxic:** a transaction (or forgotten BEGIN in a console) open for hours pins old snapshots, so VACUUM can’t clean *anything* newer — bloat everywhere.
- **In Postgres, an UPDATE is physically an insert** — and by default must touch every index (HOT updates can avoid this when no indexed column changed and the page has room).
- (MySQL/InnoDB does MVCC differently — old versions live in undo logs and are rebuilt on demand — same concept, different housekeeping.)

8 Part 6: Isolation Levels and Their Anomalies

Perfect isolation (as if transactions ran one at a time) costs performance, so SQL defines levels that permit specific, named anomalies:

The anomalies:

- **Dirty read:** seeing uncommitted data from another transaction. (Effectively never allowed in Postgres.)
- **Non-repeatable read:** read a row twice in one transaction, get different values because someone committed in between.
- **Phantom read:** rerun a *query* and get new/removed rows.
- **Lost update:** two transactions read the same value, both compute, both write — one overwrites the other. (Classic: two requests both read stock = 1 and both sell the item.)
- **Write skew:** two transactions each check a condition, both pass, both write — and *together* violate the rule (two doctors both go off-call because “someone else is on call”).

The levels (Postgres reality):

Level	Prevents	How it works	Use
Read Committed (default)	dirty reads	fresh snapshot <i>per statement</i>	fine for most apps — but not for read-modify-write!
Repeatable Read	+ non-repeatable & phantom (in PG)	one snapshot <i>per transaction</i>	reports, multi-step reads needing consistency

Serializable	everything incl. write skew	snapshot + conflict detection; losers get serialization errors	money, inventory, invariants — must retry on error
--------------	--------------------------------	---	--

Solving the lost update in practice (know all four):

1. **Atomic update:** UPDATE stock SET qty = qty - 1 WHERE id = 5 AND qty > 0; — the check and write are one statement.
2. **Pessimistic lock:** SELECT ... FOR UPDATE — lock the row while you decide.
3. **Optimistic concurrency:** version column; UPDATE ... WHERE id = 5 AND version = 7 — if 0 rows updated, someone beat you; retry.
4. **SERIALIZABLE** — the database detects the conflict; you retry the transaction.

9 Part 7: Locks and Deadlocks

Even with MVCC, writes lock: row locks for UPDATE/DELETE/SELECT FOR UPDATE, table-level locks for DDL (ALTER TABLE wants exclusive access — beware migrations on hot tables; they can queue behind one long query and then block *everyone*).

Deadlock in four lines:

Txn A: lock row 1 ... wants row 2

Txn B: lock row 2 ... wants row 1 --> cycle; DB kills one (deadlock error)

Prevention: **lock in a consistent order** (e.g., always by ascending id), keep transactions short, and handle deadlock errors with a retry. Also know SELECT FOR UPDATE SKIP LOCKED — the standard trick for building job queues in SQL (workers grab unlocked rows and skip contested ones).

10 Part 8: Storage Engine Alternatives — B-trees vs. LSM-trees (Advanced)

Postgres/InnoDB update B-tree pages in place (read-optimized). Write-heavy systems (RocksDB, Cassandra, LevelDB) use **LSM-trees**: writes go to an in-memory buffer, get flushed as sorted immutable files, and background *compaction* merges files. Writes become sequential and fast; reads may check several files (helped by Bloom filters); compaction consumes background I/O. Rule of thumb: B-trees favor reads and predictable latency; LSMs favor massive write throughput. Also know **clustered tables**: in InnoDB the primary key *is* the table (rows live inside the PK B-tree; secondary indexes point at PK values) — this is why small sequential primary keys matter more in MySQL than in Postgres.

11 Part 9: Common Mistakes, Best Practices, FAQs

11.1 Common mistakes

1. **Guessing instead of EXPLAIN ANALYZE** — the plan is the ground truth; everything else is folklore.
2. **Indexing every column “just in case”** — write amplification and cache pollution; index your query patterns.
3. **Functions on indexed columns in WHERE** — `lower(email) = ...` skips the plain index; use a functional index or restructure.
4. **Violating leftmost-prefix** — an index on (a, b) does nothing for `WHERE b = ...`.
5. **Long-running / idle-in-transaction sessions** — snapshot pinning, bloat, blocked vacuums, blocked DDL.
6. **Read-modify-write across two statements at Read Committed** — the lost-update classic; use atomic updates, `FOR UPDATE`, versions, or `SERIALIZABLE+retry`.
7. **SELECT * on wide tables** — drags dead-weight columns through pages, memory, and network; also forfeits index-only scans.
8. **Migrations without lock awareness** — `ALTER TABLE/index` builds on hot tables at noon; prefer `CREATE INDEX CONCURRENTLY` and off-peak windows.
9. **Stale statistics after bulk loads** — the planner misestimates by 1000x; run `ANALYZE`.
10. **N+1 queries from ORMs** — 1 query for the list + N for children; use joins/eager loading.

11.2 Best practices

- Make `EXPLAIN ANALYZE` reflexive; keep an eye on `pg_stat_statements` for your true top queries.
- Design composite indexes from real predicates: equality columns first, range last; consider covering (`INCLUDE`) for hot reads.
- Keep transactions short; retry on serialization failures and deadlocks; lock in consistent order.
- Choose the weakest isolation level that preserves your invariants — and *know* your invariants.
- Monitor autovacuum, dead tuples, index bloat and usage; drop unused indexes.
- Use connection pooling (Postgres connections are processes; they’re expensive).
- Batch bulk writes; use `COPY` for loads; `ANALYZE` afterward.

11.3 FAQs

Q: Why doesn’t the database use my index? A: Usually one of: the condition isn’t selective enough (seq scan is genuinely cheaper), a function/typecast wraps the column, the leftmost-prefix rule isn’t satisfied, or statistics are stale. `EXPLAIN` tells you which.

Q: Why is my query fast the second time? A: The first run pulled pages from disk into the buffer pool; the second run hits RAM.

Q: How many indexes are too many? A: When writes hurt or indexes go unused.

Every index taxes every write. Audit usage stats; a handful of well-designed composite indexes beats a dozen single-column ones.

Q: COUNT(*) is slow on a huge table — why? A: MVCC means visibility is per-transaction, so Postgres must actually check row versions; there's no free universal counter. Use estimates (reltuples), a counter table, or index-only scans where possible.

Q: When do I actually need SERIALIZABLE? A: When correctness depends on a condition across rows (uniqueness you enforce in code, "at least one on-call doctor," non-overlapping bookings). Wrap it with retry logic.

Q: Postgres vs. MySQL — internals-wise? A: Same big ideas (B-trees, WAL/redo, MVCC). Notable differences: InnoDB clusters rows in the primary key and keeps old versions in undo logs (no VACUUM, but purge lag instead); Postgres heaps rows and vacuums dead tuples. Both reward the exact skills in this guide.

12 Final Cheat Sheet (One-Page Revision)

Physical layer: disks are read in pages (8 KB PG / 16 KB InnoDB) → locality rules; buffer pool = page cache in RAM; narrow tables & hot pages = speed.

Indexes: B-tree = sorted leaves + signpost levels → 3-4 page reads even for billions of rows. Helps =, <, >, BETWEEN, IN, ORDER BY, prefix-LIKE. Broken by functions on the column and %x patterns. Composite = leftmost-prefix; equality first, range last. Covering index → index-only scan. Every index taxes every write.

Planner: SQL says what, planner picks how, using statistics + selectivity. Seq scan is *correct* for large fractions. EXPLAIN ANALYZE: hunt big-table seq scans with selective filters, 10x row misestimates (→ ANALYZE), Filter-after-fetch, disk sorts. Joins: nested loop (small outer + indexed inner), hash (big equality joins), merge (pre-sorted).

Transactions: ACID; WAL = log first, data later → crash recovery, replication, PITR.

MVCC: updates create versions (xmin/xmax); snapshots give consistent reads; readers & writers never block each other. Cost: dead tuples → VACUUM; long transactions pin snapshots → bloat.

Isolation: Read Committed (default; per-statement snapshot) < Repeatable Read (per-txn snapshot) < Serializable (detects write skew; retry on error). Lost-update fixes: atomic UPDATE • FOR UPDATE • version column • SERIALIZABLE+retry.

Locks: writers block writers on the same row; DDL takes table locks (migrate carefully, CONCURRENTLY); deadlocks → consistent lock order + retry; SKIP LOCKED = job queues.

Advanced: B-tree (read-optimized, in-place) vs. LSM (write-optimized, compaction); InnoDB clusters by primary key.

The one rule: *The database is not magic — it is pages, trees, versions, and a log. When something is slow or wrong, one of those four is telling you why; EXPLAIN ANALYZE is how you listen.*